

TrialCheck

Interview Defense Document v3.0

FAANG-Standard Complete Edition

Platform-Agnostic A/B Experiment Readout Auditing

Sidharth Kriplani | github.com/SidharthKriplani/trialcheck | PyPI: trialcheck

Label	Meaning
[IMPLEMENTED]	Code exists, runs, produces verified artifact
[SIMULATED]	Deliberately synthetic scenario; simulation logic is real code
[PARTIALLY VERIFIED]	Code exists but verification is internal consistency only
[FUTURE]	Not built — explicitly absent from the repo

QUICK REFERENCE

Metric	Value
Total checks	7 (v0.2.0) + 2 new in v0.3.0 = 9
Checks that can FAIL	1 — SRM only
Checks that WARN	8
Dependencies	0 — pure Python stdlib
PyPI package	<code>pip install trialcheck</code>
Test cases	27
Demo scenarios	4 (clean pass, SRM fail, peeking warn, guardrail harm)
Output formats	JSON / Markdown / HTML

Metric	Value
Current version	v0.3.0
Core entry point	TrialCheck(experiment).run()
SRM method	chi-squared df=1, exact p via erfc
Continuous metric test	Welch t-test, Welch-Satterthwaite df
Primary metric test	Two-proportion z-test, pooled SE

STOC Table of Contents

- S0** Cover + Quick Reference
 - S1** The Problem — First Principles
 - S2** Architecture + All 9 Checks
 - S3** Algorithm Explanations with Worked Numerical Examples
 - S4** Design Decisions (6 explicit decisions, alternatives rejected)
 - S5** Honest Positioning — IMPLEMENTED / SIMULATED / FUTURE
 - S6** Q&A; Battery — 32 Pairs (FAANG Standard)
 - S7** Failure Modes
 - S8** Scalability
 - S9** Cross-Project Connections
 - S10** Resume Claim Audit
 - S11** Study Plan + Do Not Say This
-

How to Use This Document

S1-S5 build the statistical foundation. S6 is the Q&A; battery — 32 pairs covering all common FAANG interview angles. S7-S9 cover adversarial questions. S10-S11 are preparation tools. The statistical algorithms in S3 (SRM derivation, Welch's t, two-proportion z-test) are the most important section — an interviewer asking 'walk me through your SRM check' expects you to derive the chi-squared statistic and interpret the p-value, not just say 'I use a chi-squared test'.

At FAANG E5 level, the expected answer format for every design decision in S4 is: state the decision, state the alternative that was considered, explain why the alternative was rejected. One-sentence answers will not pass. Three-sentence answers with all three components will.

Before your interview: derive the Welch-Satterthwaite df formula from scratch. If $A = s_t^2/n_t$ and $B = s_c^2/n_c$, explain why $df = (A+B)^2 / (A^2/(n_t-1) + B^2/(n_c-1))$ makes the test more conservative when one group is noisier. If you can explain this, you can handle any statistics question the interviewer throws.

S1 The Problem — First Principles

1.1 The Checklist That Lives in People's Heads

The problem is not that data scientists do not know what to check before shipping an experiment. Senior DS have a consistent mental checklist: Did assignment work correctly? Was this called early? Is the lift real and large enough to matter? Did any guardrail move in the wrong direction? Were groups balanced before the test started? Most practitioners with 3+ years of A/B experience could recite this list.

The problem is that this checklist lives in runbooks, reviewer notes, and institutional memory — applied inconsistently, skipped under deadline pressure, and invisible to junior practitioners who have not learned it yet. A fast-moving team will ship on a p-value alone. A careful one catches the SRM at review, the peeking three weeks later, and the guardrail regression only in production. TrialCheck packages that senior DS checklist into a library call.

1.2 What Goes Wrong When the Checklist Is Skipped

Sample Ratio Mismatch (SRM): Assignment was broken — the observed 53/47 split from a planned 50/50 indicates some users were not properly randomised. Every downstream statistical test is invalidated because the groups are not comparable.

Peeking: The experiment was called significant on day 7 of a planned 14-day run because the dashboard looked good. Peeking at $\alpha=0.05$ mid-experiment raises the effective false positive rate to 8-14%.

Underpowering: The experiment had $n=500$ per arm but the MDE requires $n=2,000$. The observed lift may be noise, not signal.

Practical insignificance: $p=0.001$ but lift=0.02% conversion rate improvement. Statistically significant, practically meaningless.

Guardrail regression: Primary metric improved by 4%. P99 latency increased by 3.2%. The feature broke performance while improving conversion — a tradeoff that needed a stakeholder conversation, not a ship decision.

Pre-period imbalance: The two groups had different pre-experiment baseline metrics (SMD=0.15 on revenue). The observed lift may partly reflect this pre-existing difference.

1.3 What TrialCheck Is and Is Not

Claim Boundary (printed on every report): TrialCheck is an audit helper, not a decision-maker. It does NOT run experiments, assign users, replace an experimentation platform, prove causal validity, perform CUPED or sequential testing, correct for multiple comparisons across metrics, or make automatic ship/no-ship decisions.

1.4 Why Platform-Agnostic Design Matters

A/B experimentation infrastructure varies enormously across organisations: Statsig, Optimizely, LaunchDarkly, in-house systems at Meta/Google/Amazon, custom Airflow DAGs. A tool that requires any specific platform cannot be the organisational standard. TrialCheck's ExperimentSummary is a pure Python dataclass — any platform can produce one by extracting the required fields and constructing the object. The checks run entirely on the summary object, with no platform SDK installed.

The practical implication: a data scientist who uses Statsig today and moves to a company using an in-house platform next year runs the same TrialCheck, adapting only the extraction layer that builds ExperimentSummary. The audit logic never changes.

1.5 The Cost of Not Auditing — A Concrete Example

Suppose a team ships a feature that improves conversion rate by 2.1% ($p=0.03$) after a 7-day experiment. Three months later, the same team runs a retrospective and finds:

SRM was present: The actual split was 53/47 because a caching layer occasionally served cached experiences to treatment users. Every statistical test was invalid. The 2.1% lift included the selection bias from 3% more high-intent users in the treatment group.

Peeking occurred: The experiment was called on day 7 of a planned 14-day run because someone checked the dashboard and the numbers looked good. Effective FPR was 9.2%, not 5%.

Guardrail P99 latency increased 8%: Not noticed at ship time because the dashboard only showed P50 latency.

The feature shipped. Three months and four team-weeks of downstream development were built on a result that was likely a false positive from broken randomisation. TrialCheck would have flagged FAIL on SRM before the experiment was read out.

S2 Architecture + All 9 Checks

2.1 Entry Point and Data Flow

The public API: `TrialCheck(experiment).run()`. The `experiment` is an `ExperimentSummary` dataclass — a plain Python object with assignment counts, metric outcomes, optional guardrails and covariates. No platform-specific objects. No database connections. The `run()` method executes all checks, aggregates status under `FAIL > WARN > PASS`, and returns a `TrialReport` with per-check verdicts and interpretations.

```
from trialcheck import TrialCheck, ExperimentSummary, GuardrailMetric
from trialcheck.models import VariantSummary
experiment = ExperimentSummary(
    experiment_id="checkout_v2", metric_name="conversion_rate",
    control=VariantSummary(name="control", n=50120, conversions=1500),
    treatment=VariantSummary(name="treatment", n=49880, conversions=1620),
    expected_control_ratio=0.5, alpha=0.05, planned_duration_days=14,
    actual_duration_days=11, guardrails=[ GuardrailMetric( name="p99_latency_ms",
    control_value=234.1, treatment_value=241.7, bad_direction="increase",
    tolerance=0.02, ), ], )
report = TrialCheck(experiment).run()
print(report.overall_status.value) # FAIL / WARN / PASS
print(report.interpretation)
```

2.2 Check Architecture Overview

#	Check	Group	Method	Fires When
1	SRM	Randomization	chi-sq df=1, exact p via erfc	Observed split deviates from planned ratio
2	Pre-period balance	Randomization	SMD per covariate, pooled SD	Any covariate SMD >= 0.1
3	Primary metric	Statistical	Two-proportion z-test, pooled SE	p > alpha or missing counts
4	Continuous metric	Statistical	Welch t-test, W-S df	p > alpha for mean comparison
5	Peeking risk	Statistical	Duration ratio + interim looks	Called before 80% of planned duration
6	Practical significance	Practical	Observed lift vs business threshold	Lift real but below practical threshold
7	Guardrail movement	Practical	Direction check + tolerance band	Guardrail moves bad direction beyond tolerance
8	Min sample size	Statistical	Required n for MDE at given power	Actual n < required n
9	Novelty effect	Behavioral	Runtime vs novelty washout window	Runtime < novelty_effect_days threshold

2.3 Sample Ratio Mismatch (SRM) [IMPLEMENTED — Only FAIL check]

SRM occurs when the actual assignment ratio differs from the planned ratio in a way that cannot be explained by chance alone. TrialCheck computes: $\chi^2_{sq} = (n_c - n_{exp_c})^2 / n_{exp_c} + (n_t - n_{exp_t})^2 / n_{exp_t}$, where $n_{exp_c} = total_n * expected_control_ratio$. The p-value is derived via $erfc(\sqrt{\chi^2_{sq} / 2})$. SRM is the only FAIL because it invalidates the randomisation assumption that every other statistical test depends on. An experiment with SRM cannot be salvaged.

2.4 Pre-Period Balance [IMPLEMENTED]

Even with correct assignment ratios, groups might have different baseline distributions. For each covariate: $SMD = (mean_treatment - mean_control) / pooled_std$. $SMD \geq 0.1$ on any covariate is flagged WARN — not FAIL because imbalance is often addressable by CUPED rather than requiring re-randomisation.

2.5 Primary Metric Significance [IMPLEMENTED]

Two-proportion z-test for binary outcomes (conversion rate, click rate). $z = (p_t - p_c) / SE_pooled$, where $SE_pooled = \sqrt{p_bar(1-p_bar)(1/n_t + 1/n_c)}$ and p_bar is the pooled conversion rate. Non-significant result is WARN not FAIL — non-significance may mean underpowered, not no effect.

2.6 Continuous Metric Significance [IMPLEMENTED]

Welch t-test for continuous outcomes (revenue per user, session duration). Does not assume equal variances — the correct assumption for real A/B data. Welch-Satterthwaite df: $(s_t^2/n_t + s_c^2/n_c)^2 / ((s_t^2/n_t)^2/(n_t-1) + (s_c^2/n_c)^2/(n_c-1))$. Produces more conservative p-values than Student's t-test when variances are unequal.

2.7 Peeking Risk [IMPLEMENTED]

Two signals: (1) $actual_duration / planned_duration < 0.8$ — called before 80% of planned duration; (2) $interim_looks \geq 3$ with no sequential testing correction. Either condition produces WARN. Peeking at $\alpha=0.05$ mid-experiment raises the effective false positive rate to 8-14% depending on how many interim looks occurred.

2.8 Practical Significance [IMPLEMENTED]

Checks whether $observed_lift \geq practical_significance_threshold$ (configurable, default 1% relative lift). An experiment that is statistically significant but practically negligible ($lift=0.01\%$) is flagged WARN. This is decision support for the ship/no-ship call, not a statistical test.

2.9 Guardrail Movement [IMPLEMENTED]

For each guardrail metric: checks (1) did the metric move in the bad direction, and (2) did the movement exceed guardrail_tolerance (default 2% relative). Both conditions must hold for WARN to fire — prevents measurement noise from triggering false alarms.

2.10 Minimum Sample Size [IMPLEMENTED — v0.3.0]

Computes required sample size for the observed effect size at configured power (default 0.80) and alpha. If actual sample < required, flags WARN for underpowering. The observed lift may be noise in an underpowered experiment.

2.11 Novelty Effect [IMPLEMENTED — v0.3.0]

Flags experiments where runtime < novelty_effect_days threshold (default 7 days). Early uplift may reflect user curiosity rather than sustained behaviour change that will persist post-launch.

2.12 Test Suite Structure

The test suite validates every check against known inputs and scipy reference values. 27 tests across 9 categories:

Category	Tests	What Is Validated
SRM detection	4	Perfect split PASS; extreme SRM FAIL; borderline split PASS; non-50/50 planned ratio PASS
Pre-period balance	3	Balanced covariates PASS; SMD=0.12 WARN; missing covariates WARN
Primary metric (z-test)	3	Significant result PASS; non-significant WARN; zero conversions edge case
Continuous metric (Welch)	3	Significant Welch PASS; high treatment variance (W-S df correctness); non-significant WARN
Peeking risk	3	Full duration PASS; called at 70% WARN; interim_looks >= 3 WARN
Practical significance	2	Lift above threshold PASS; lift below threshold WARN
Guardrail movement	3	No movement PASS; bad direction exceeds tolerance WARN; within tolerance PASS
Min sample size	3	Adequate n PASS; underpowered WARN; no MDE provided (missing input)
Novelty effect	3	Runtime >= 7 days PASS; runtime < 7 days WARN; novelty_effect_days overridden

Each statistical test category includes a scipy cross-validation: TrialCheck's computed p-value is compared against `scipy.stats` (`chi2_contingency`, `ttest_ind`, `proportions_ztest`) with tolerance $1e-6$. If you are asked 'how do you verify the math?', this is the complete answer.

S3 Algorithm Explanations with Worked Numerical Examples

3.1 SRM Detection from First Principles

The chi-squared test for goodness-of-fit asks: is the observed distribution consistent with the expected distribution under the null hypothesis of correct randomisation? For a two-arm experiment with planned ratio r (typically 0.5):

```
n_expected_control = total_n * r
n_expected_treatment = total_n * (1 - r)
chi_sq = (n_c - n_exp_c)^2 / n_exp_c + (n_t - n_exp_t)^2 / n_exp_t
For df=1, the survival function is: p_value = erfc(sqrt(chi_sq / 2))
erfc is the complementary error function from Python's math module.
SRM fires when p_value < alpha (default 0.05).
```

Worked Example — SRM Computation:

```
Planned: 50/50 split, total_n = 100,000
Observed: n_control = 56,000, n_treatment = 44,000
n_exp_c = 100,000 * 0.5 = 50,000
n_exp_t = 100,000 * 0.5 = 50,000
chi_sq = (56000 - 50000)^2 / 50000 + (44000 - 50000)^2 / 50000
        = 36,000,000 / 50,000 + 36,000,000 / 50,000
        = 720 + 720 = 1440
p_value = erfc(sqrt(1440/2)) = erfc(sqrt(720)) = erfc(26.83) ≈ 0
Result: FAIL — extreme SRM. Every downstream test is invalid.
```

Worked Example — Borderline SRM:

```
total_n = 10,000, observed: n_c = 5,100, n_t = 4,900
chi_sq = (5100-5000)^2/5000 + (4900-5000)^2/5000
        = 10000/5000 + 10000/5000 = 2.0 + 2.0 = 4.0
p_value = erfc(sqrt(4.0/2)) = erfc(sqrt(2)) = erfc(1.414) ≈ 0.157
Result: 0.157 > 0.05 -> PASS. A 51/49 split is random noise at this scale.
```

3.2 Welch's t-test and the Welch-Satterthwaite Degrees of Freedom

The Welch t-statistic for comparing two sample means is:

```
t = (mean_t - mean_c) / sqrt(s_t^2/n_t + s_c^2/n_c)
This is identical to Student's t but uses separate variance estimates for each group.
Student's t assumes s_t = s_c (homoscedasticity). Welch's t makes no such assumption.
```

The degrees of freedom from Welch-Satterthwaite:

```
Let A = s_t^2 / n_t and B = s_c^2 / n_c
df = (A + B)^2 / (A^2/(n_t - 1) + B^2/(n_c - 1))
Intuition: if both variances are equal (A = B), this simplifies to n_t + n_c - 2,
the same as Student's t. When A >> B (treatment has much higher variance), df is
dominated by the smaller group, producing a more conservative test.
Conservative here means higher p-values -> fewer false positives.
```

Worked Example — Welch's t-test:

Revenue experiment: control: mean_c = \$48.20, std_c = \$21.30, n_c = 5,000 treatment: mean_t = \$51.80, std_t = \$28.40, n_t = 5,000 (Treatment has higher std – high spenders in treatment arm create right tail) $A = 28.40^2 / 5000 = 806.56 / 5000 = 0.16131$ $B = 21.30^2 / 5000 = 453.69 / 5000 = 0.09074$ $t = (51.80 - 48.20) / \sqrt{0.16131 + 0.09074} = 3.60 / \sqrt{0.25205} = 3.60 / 0.5020 = 7.17$ $df = (0.16131 + 0.09074)^2 / (0.16131^2/4999 + 0.09074^2/4999) = 0.25205^2 / (0.02602/4999 + 0.00823/4999) = 0.063529 / (0.000068) = 9,342$ (much less than $n_c + n_t - 2 = 9,998$) With $t=7.17$ and $df=9,342$: $p < 0.001 \rightarrow$ PASS (significant). Student's t would give $df=9,998 \rightarrow$ slightly more optimistic p-value. Welch's is more conservative: fewer false positives.

3.3 Two-Proportion Z-Test for Primary Metric

For binary outcomes (conversion rate):

```
p_c = conversions_c / n_c p_t = conversions_t / n_t p_bar = (conversions_c + conversions_t) / (n_c + n_t) # pooled rate SE_pooled = sqrt(p_bar * (1 - p_bar) * (1/n_t + 1/n_c)) z = (p_t - p_c) / SE_pooled p_value = 2 * (1 - Phi(|z|)) # two-tailed where Phi is the standard normal CDF.
```

Worked Example:

```
n_c = 50,120, conversions_c = 1,500 -> p_c = 0.02993 n_t = 49,880, conversions_t = 1,620 -> p_t = 0.03248 p_bar = (1500 + 1620) / (50120 + 49880) = 3120 / 100000 = 0.03120 SE = sqrt(0.03120 * 0.96880 * (1/49880 + 1/50120)) = sqrt(0.030224 * 0.00003999) = sqrt(0.00000120859) = 0.001099 z = (0.03248 - 0.02993) / 0.001099 = 0.00255 / 0.001099 = 2.32 p_value = 2 * (1 - Phi(2.32)) = 2 * 0.0102 = 0.0204 0.0204 < 0.05 -> PASS (statistically significant).
```

3.4 Peeking Risk — Why It Inflates False Positive Rate

Peeking is calling an experiment early when it looks significant. At $\alpha=0.05$ with a single look at the end, FPR = 5% by definition. If you peek at 50% duration and call it if significant: you have two chances to see a false positive, compounding the error rate. With 3 interim looks at equal intervals: effective FPR ~ 8-14%. TrialCheck checks two signals: (1) $\text{actual_duration} / \text{planned_duration} < 0.8$ — called before 80% of planned duration; (2) $\text{interim_looks} \geq 3$ with no sequential testing correction applied. The check documents the risk, it does not compute the adjusted alpha — that requires knowing the interim look schedule, which TrialCheck does not have.

3.5 SMD for Pre-Period Balance

```
SMD = (mean_treatment - mean_control) / pooled_std pooled_std = sqrt((std_c^2 + std_t^2) / 2) SMD is a standardised measure: the difference in standard deviation
```

units. This makes it comparable across covariates with different scales (age in years vs revenue in dollars vs session duration in seconds). Cohen conventions: small = 0.2, medium = 0.5, large = 0.8. TrialCheck uses 0.1 as the warning threshold – below 'small' – because even small pre-existing differences can compound with treatment effects.

Worked Example — Pre-Period Balance:

Experiment: checkout_v3 Covariate: pre-experiment purchase count (7-day window)
control: mean=2.40, std=1.85, n=25,000 treatment: mean=2.55, std=1.92, n=25,000
 $\text{pooled_std} = \sqrt{((1.85^2 + 1.92^2) / 2)} = \sqrt{(3.4225 + 3.6864) / 2} = \sqrt{3.5545} = 1.885$
 $\text{SMD} = (2.55 - 2.40) / 1.885 = 0.15 / 1.885 = 0.0796$
 $0.0796 < 0.10 \rightarrow \text{PASS}$. Groups are well-balanced on purchase history. If treatment mean were 2.60: $\text{SMD} = (2.60 - 2.40) / 1.885 = 0.106 \rightarrow \text{WARN}$. Interpretation: treatment group had 0.2 more purchases pre-experiment, which might inflate observed treatment lift on conversion.

3.6 Minimum Sample Size Calculation

The minimum sample size check computes n_{required} for a given MDE, alpha, and power:

$n_{\text{required}} = 2 * ((z_{\alpha/2} + z_{\beta})^2 * p * (1-p)) / \text{MDE}^2$ For alpha=0.05, power=0.80: $z_{\alpha/2} = 1.96$ (from standard normal, two-tailed at 0.025) $z_{\beta} = 0.842$ (from standard normal, one-tailed at 0.20) Example: MDE=1% absolute (from 4% to 5%), $p_{\text{baseline}}=0.04$ $n_{\text{required}} = 2 * ((1.96 + 0.842)^2 * 0.04 * 0.96) / 0.01^2 = 2 * (7.853 * 0.0384) / 0.0001 = 2 * 0.3015 / 0.0001 = 6,030$ per arm If actual $n_c = 3,500 < 6,030$: WARN (underpowered). An underpowered experiment that shows $p < 0.05$ may be a false positive: the observed effect is larger than the true effect by noise.

3.7 Implementation Internals — How the Check Pipeline Works

Understanding the TrialCheck source code at function level is necessary for 'walk me through your codebase' questions. Every check follows the same signature:

```
def some_check( experiment: ExperimentSummary, ) -> CheckResult: """Returns CheckResult with status, detail, recommendation, evidence.""" if experiment.some_required_field is None: return CheckResult(status=CheckStatus.WARN, detail='Required field missing', recommendation='Provide field to run this check', evidence={}) # ... check logic ... return CheckResult(status=..., detail=..., recommendation=..., evidence=...)
```

Unlike GoldenSetAuditor (which has a four-level FAIL/WARN/INSUFFICIENT_INPUT/PASS hierarchy), TrialCheck uses three levels: FAIL/WARN/PASS. Missing data returns WARN in TrialCheck because in the experiment context, missing data is itself a signal — an experiment report that does not

include covariate data should warn the reviewer, not silently pass.

The run() method in TrialCheck:

```
class TrialCheck: def __init__(self, experiment: ExperimentSummary): self.experiment = experiment def run(self) -> TrialReport: checks = [ srm_check, pre_period_balance_check, primary_metric_check, continuous_metric_check, peeking_risk_check, practical_significance_check, guardrail_movement_check, minimum_sample_size_check, novelty_effect_check, ] results = [check(self.experiment) for check in checks] overall = self._aggregate(results) return TrialReport(overall_status=overall, check_results=results, claim_boundary=CLAIM_BOUNDARY, interpretation=self._interpret(results) ) def _aggregate(self, results): statuses = {r.status for r in results} if CheckStatus.FAIL in statuses: return CheckStatus.FAIL if CheckStatus.WARN in statuses: return CheckStatus.WARN return CheckStatus.PASS
```

Key design properties: (1) All checks run regardless of SRM — even if SRM is FAIL, all other checks still run and their results appear in the report. This gives reviewers full visibility. (2) The claim boundary is a module-level constant (CLAIM_BOUNDARY) referenced by the TrialReport constructor — it cannot be accidentally omitted. (3) The check list in run() is ordered by check category: randomisation first (SRM, balance), statistical tests second, practical and behavioral checks last.

S4 Design Decisions — 6 Explicit Choices, Alternatives Rejected

4.1 Zero External Dependencies

Decision: TrialCheck has no runtime dependencies. All statistical tests are implemented from scratch using Python's math module (erfc, sqrt, exp) and the standard library. ML teams cannot always control their dependency environment. A tool that requires scipy, statsmodels, or numpy can fail to install in constrained environments. Zero dependencies means TrialCheck works in any Python 3.10+ environment, including restricted cloud execution environments. Correctness is validated against scipy reference values in tests — scipy is a test dependency, not a runtime dependency.

Alternative rejected: Alternative rejected: use scipy.stats for all statistical tests. Rejected because scipy adds ~50MB installation overhead, has version-specific API changes, and is unavailable in some constrained ML deployment environments.

4.2 FAIL Reserved for SRM Only

Decision: SRM is the only FAIL-level finding because it is the only condition that invalidates all other checks. An experiment with SRM cannot be salvaged: the groups being compared are not comparable, so no amount of statistical adjustment produces trustworthy results. Every other finding — peeking, practical insignificance, guardrail movement — produces WARN because each is individually addressable or contextually interpretable.

Alternative rejected: Alternative rejected: FAIL for guardrail regression. A guardrail regression is serious but is a tradeoff decision, not a statistical invalidity. The product team might ship accepting a known latency increase for a large enough conversion lift. That decision requires human judgment, not a binary FAIL gate.

4.3 Platform-Agnostic ExperimentSummary Design

Decision: TrialCheck accepts a plain ExperimentSummary dataclass — a simple Python object with numbers, not platform-specific objects. This means it works regardless of whether experiments are run on Optimizely, Split.io, Statsig, an internal platform, or a hand-rolled randomisation system. The input schema is the smallest set of numbers needed to run all checks.

Alternative rejected: Alternative rejected: native Optimizely/Statsig integration. Rejected because it would limit TrialCheck to teams using specific platforms and create vendor lock-in. The platform-agnostic design means any team can wrap their platform's output in an ExperimentSummary and audit it.

4.4 Welch's t-test Instead of Student's t-test for Continuous Metrics

Decision: Welch's t-test does not assume equal variances between groups (homoscedasticity). In A/B tests, this assumption is almost always violated: the treatment group's metric variance is affected by the treatment itself. For revenue experiments, high-spending users in treatment create a longer right tail. For latency experiments, the treatment might affect p99 differently from mean latency. Welch's t-test is uniformly more appropriate for A/B test data.

Alternative rejected: Alternative rejected: Student's t-test. Rejected because it assumes equal variances, which is wrong for almost all real A/B tests. Using Student's t produces lower p-values (more optimistic significance) when the treatment group has higher variance — increasing false positive rate.

4.5 Claim Boundary on Every Report

Decision: TrialCheck's claim boundary (it does not make ship/no-ship decisions) is printed on every output format and cannot be suppressed. This prevents reports from being used to automatically ship features without human review. TrialCheck surfaces risks; it does not make decisions. The ship call requires understanding business context that TrialCheck does not have.

Alternative rejected: Alternative rejected: add a recommended action field (ship/hold/investigate). Rejected because a recommended action field would be treated as a decision in practice, removing human judgment from the loop.

4.6 No Automatic Bonferroni Correction

Decision: TrialCheck does not automatically apply multiple comparison correction across metrics. Teams that want Bonferroni correction should manually adjust alpha before passing it. Rationale: applying Bonferroni to the primary metric check is overly conservative — you specified one primary metric; adjusting for additional secondary metrics compounds the correction inappropriately. The right correction depends on the experiment design, not on the number of metrics TrialCheck happens to be checking.

Alternative rejected: Alternative rejected: auto-apply Bonferroni for all metric checks. Rejected because Bonferroni is often too conservative for correlated metrics and would flag too many legitimate results. Teams should apply the correction they designed their experiment around.

4.7 Three-Level Status Hierarchy vs Binary Pass/Fail

Decision: FAIL / WARN / PASS is a three-level hierarchy. FAIL is reserved for SRM — the only condition that structurally invalidates the experiment. WARN covers all addressable problems: peeking, guardrail movement, pre-period imbalance, underpowering. PASS means all checks ran and found nothing. A binary system would either block experiments on warnings (too restrictive, causes alert fatigue) or miss them (false safety).

Alternative rejected: Four-level hierarchy like GoldenSetAuditor's (FAIL/WARN/INSUFFICIENT_INPUT/PASS). TrialCheck uses WARN for cases where checks could not run (missing data) — because in experiment context, missing data is itself a quality signal. An experiment with no interim_looks count provided should warn, not return INSUFFICIENT_INPUT.

4.8 scipy as Test Ground Truth Only, Not Runtime Dependency

Decision: All statistical implementations (chi-squared, Welch's t, z-test) are pure Python using only `math.erfc` from `stdlib`. `scipy` is installed only in the test environment and used exclusively as reference values to validate `TrialCheck`'s math. This keeps `TrialCheck`'s runtime dependencies at zero.

Alternative rejected: Use `scipy.stats` at runtime. Rejected because adding `scipy` as a runtime dependency would create version conflict risks in environments that have pinned `scipy` for numerical work. The zero-dependency design means `TrialCheck` installs in any Python environment without conflict. The validation approach (`scipy` as test ground truth) is actually stronger: it proves the math matches a trusted reference implementation.

4.9 ExperimentSummary as a First-Class Documentation Object

Decision: `ExperimentSummary` is a dataclass with explicit field names and optional fields that `WARN` when absent. This design means creating an `ExperimentSummary` forces the user to think through every dimension of their experiment: What was the planned split? How many days was it planned to run? What were the guardrails? These questions, forced by the constructor, catch incomplete experiment documentation before the audit runs.

Alternative rejected: Accept a dictionary with free-form keys. A dict with arbitrary keys would silently ignore missing fields (no `WARN`). The typed dataclass makes missing fields explicit — optional fields return `WARN`, not silently pass. The constructor doubles as a documentation template for what a well-documented experiment should contain.

S5 Honest Positioning — IMPLEMENTED / SIMULATED / FUTURE

Feature	Status	Evidence
SRM detection (chi-sq via <code>erfc</code>)	[IMPLEMENTED]	<code>stats.py</code> ; validated vs <code>scipy.stats.chi2_contingency</code>
Pre-period balance (SMD)	[IMPLEMENTED]	<code>checks.py</code> ; validated vs manual computation
Primary metric (two-proportion z-test)	[IMPLEMENTED]	<code>stats.py</code> ; validated vs <code>scipy.stats.proportions_ztest</code>
Continuous metric (Welch t-test)	[IMPLEMENTED]	<code>stats.py</code> ; validated vs <code>scipy.stats.ttest_ind(equal_var=False)</code>
Peeking risk check	[IMPLEMENTED]	Duration ratio + interim look count
Practical significance check	[IMPLEMENTED]	lift vs <code>practical_threshold</code>

Feature	Status	Evidence
Guardrail movement check	[IMPLEMENTED]	Direction check + tolerance band
Min sample size check (v0.3.0)	[IMPLEMENTED]	MDE power analysis in checks.py
Novelty effect check (v0.3.0)	[IMPLEMENTED]	Runtime vs novelty_effect_days threshold
4 demo scenarios	[IMPLEMENTED]	scripts/generate_demo_reports.py
27 tests	[IMPLEMENTED]	tests/ directory in repo
JSON/Markdown/HTML output	[IMPLEMENTED]	write_report() in io.py
Zero dependencies	[IMPLEMENTED]	no requirements.txt runtime deps
CUPED variance reduction	[FUTURE]	Roadmap; pre-processing hook not built
Sequential testing (mSPRT)	[FUTURE]	Roadmap; would require mSPRT implementation
Multiple comparison correction (Holm)	[FUTURE]	Roadmap item; teams configure alpha manually for now
CSV batch audit mode	[FUTURE]	Single experiment only currently

FAANG interview note: Every statistical implementation in TrialCheck is validated against scipy reference values in the test suite. When asked 'how do you know the math is right?', the answer is: `scipy.stats.chi2_contingency`, `ttest_ind(equal_var=False)`, and `proportions_ztest` are used as ground truth in tests, not as runtime dependencies.

5.2 Version History — What Changed Between Releases

Version	Key Additions	Rationale
v0.1.0	SRM check + primary metric significance (binary)	Minimum viable experiment audit: the two checks that matter most. SRM catches broken randomisation; significance catches whether results are noise.
v0.2.0	Welch t-test for continuous metrics; peeking check; practical significance; guardrail movement	Covers the full analyst checklist for continuous outcomes and the three most common methodology failures (peeking, practical vs statistical, guardrail regression).
v0.3.0	Pre-period balance (SMD); minimum sample size; novelty effect check; 4 demo scenarios	Pre-period balance catches selection bias. Min sample size and novelty effect round out the 9-check audit suite. Demos make the tool usable without custom data.
v1.0 (planned)	CUPED variance reduction; sequential testing (mSPRT); multiple comparison correction	Moves from audit to analysis assistance. Requires statistical methodology decisions that are currently out of scope.

5.3 Complete Audit Walkthrough — Scenario: Guardrail Harm

Let us trace through the guardrail_harm scenario to see how TrialCheck runs all 9 checks:

```
Inputs: experiment_id = 'feature_v3_checkout' control: n=25,100, conversions=1,200  
(p_c = 0.0478) treatment: n=24,900, conversions=1,325 (p_t = 0.0532) planned_split =  
0.5, actual split = 50.2% / 49.8% planned_duration = 14 days, actual = 14 days  
interim_looks = 1 practical_threshold = 0.01 (1%) guardrails: revenue/user  
control=$47.20, treatment=$45.20 (bad_direction=decrease, tol=0.02) refund_rate  
control=0.042, treatment=0.089 (bad_direction=increase, tol=0.05)
```

```
Check-by-check: 1. SRM: chi_sq = (25100-25000)^2/25000 + (24900-25000)^2/25000 = 0.4  
+ 0.4 = 0.8 p = erfc(sqrt(0.4)) = erfc(0.632) ≈ 0.53 -> PASS 2. Pre-period balance:  
no covariates provided -> WARN (covariates missing) 3. Primary metric: z =  
(0.0532-0.0478)/SE_pooled. p_bar=0.0505, SE = sqrt(0.0505*0.9495*(1/25100+1/24900))  
= 0.00309 z = 0.0054/0.00309 = 1.75, p = 0.080 -> WARN (p > alpha=0.05) 4.  
Continuous metric: not provided -> PASS (no continuous metric configured) 5.  
Peeking: actual=14/planned=14 -> ratio=1.0 >= 0.8. interim_looks=1 < 3 -> PASS 6.  
Practical significance: (0.0532-0.0478)/0.0478 = 11.3% lift -> PASS (> 1%) 7.  
Guardrail – revenue/user: (45.20-47.20)/47.20 = -4.24%. bad_direction=decrease.  
Change exceeds tolerance 2%. -> WARN Guardrail – refund_rate: (0.089-0.042)/0.042 =  
+112%. bad_direction=increase. Change exceeds tolerance 5%. -> WARN 8. Min sample  
size: n=25,000 per arm, MDE=1%. n_required computed by power analysis. PASS. 9.  
Novelty effect: runtime=14 days >= novelty_effect_days=7 -> PASS Aggregation: WARN  
(from checks 2, 3, 7). Overall status = WARN. Interpretation: 'Conversion lift not  
significant (p=0.08). Two guardrails degraded: revenue/user -4.24% (exceeds 2%  
tolerance) and refund_rate +112%. Human review required.'
```

S6 Q&A; Battery — 32 Pairs (FAANG Standard)

6.1 Architecture and Statistical Method Questions

6.1 Core Architecture and Statistical Methods (Q1–Q5)

Q: 1. Walk me through what happens when I call `run_audit()` on an `ExperimentSummary`

Note: the method is `TrialCheck(experiment).run()` in v0.3.0. Calling `run()` triggers all 9 checks sequentially. Each check is a pure function that takes the `ExperimentSummary` and returns a `CheckResult` with status, detail, recommendation, and evidence dict. SRM runs first — if SRM is FAIL, the other checks still run (their results are included in the report) but the overall status is immediately FAIL. The status aggregation is: FAIL if any check is FAIL; else WARN if any check is WARN; else PASS. The `TrialReport` includes an interpretation string summarising the findings in plain English.

Q: 2. What is SRM and why does it cause FAIL when everything else causes WARN?

Sample Ratio Mismatch occurs when the actual assignment ratio (e.g., 56/44) differs significantly from the planned ratio (50/50) by more than random chance. SRM indicates broken randomisation: some users were not properly assigned to their groups, meaning the control and treatment groups are not drawn from the same population. This invalidates every downstream statistical test — the two-proportion z-test, Welch's t-test, and all metric comparisons assume the groups are comparable. SRM is the only FAIL because it is the only condition that invalidates all other checks. Every other problem — peeking, guardrail regression, practical insignificance — is individually addressable. SRM is not: you cannot adjust your way out of broken randomisation.

Q: 3. Derive the chi-squared statistic for SRM detection from first principles

The chi-squared goodness-of-fit test asks: given the `total_n` observations and planned ratio r , is the observed split consistent with random assignment? Under the null hypothesis (correct randomisation): $n_{\text{expected_control}} = \text{total_n} * r$; $n_{\text{expected_treatment}} = \text{total_n} * (1-r)$. The test statistic is the sum of squared normalised deviations: $\text{chi_sq} = \sum_i (\text{observed_i} - \text{expected_i})^2 / \text{expected_i} = (n_c - n_{\text{exp_c}})^2 / n_{\text{exp_c}} + (n_t - n_{\text{exp_t}})^2 / n_{\text{exp_t}}$. For two arms ($df=1$), the exact p-value via the complementary error function: $p = \text{erfc}(\text{sqrt}(\text{chi_sq} / 2))$. This is exact (not chi-sq table lookup) and is implemented using `math.erfc` from Python `stdlib`. If $p < \alpha$, the deviation from planned split is statistically improbable under correct randomisation -> SRM detected.

Q: 4. What is the peeking problem and why is it hard to detect post-hoc?

Peeking is calling an experiment significant when an early interim look shows $p < 0.05$, then continuing the experiment if it does not look significant. This inflates the effective false positive rate beyond alpha. At $\alpha=0.05$: a single final look gives 5% FPR. Multiple interim looks (peeking) give 8-14% FPR depending on how many looks and when. Hard to detect post-hoc because: (1) the experiment report shows the final p-value, not the history of when it was checked. (2) The person who peeked may not document it. (3) The final p-value may be 0.03 — significant by normal standards — but was cherry-picked from a sequence of checks where it briefly dipped below 0.05. TrialCheck catches peeking via two signals: $\text{duration_ratio} < 0.8$ and $\text{interim_looks} \geq 3$.

Q: 5. Why Welch's t-test instead of Student's t-test?

Student's t-test assumes equal variances between groups (homoscedasticity). In A/B tests this assumption is almost always violated: the treatment affects the variance structure of the metric, not just its mean. Revenue example: treatment improves average order value among high-spenders while leaving low-spenders unchanged. High-spenders' revenue has higher variance in the treatment group $\rightarrow$ treatment variance $>$ control variance. Student's t-test, assuming equal variances, would use the pooled variance, underestimating the true uncertainty, and producing a lower p-value (more false positives). Welch's t-test uses separate variance estimates per group and produces a more conservative (higher) p-value. The cost is a slightly more complex df computation (Welch-Satterthwaite), which TrialCheck handles internally.

6.2 Specific Check Logic — SRM, Peeking, Balance (Q6–Q10)

Q: 6. Derive the Welch-Satterthwaite degrees of freedom formula intuitively

The formula is: $df = (A + B)^2 / (A^2/(n_t-1) + B^2/(n_c-1))$ where $A = s_t^2/n_t$ and $B = s_c^2/n_c$. Intuition: A and B are the variance-of-the-mean contributions from each group. If both groups have equal variance ($A = B$), the formula gives: $df = (2A)^2 / (A^2/(n-1) + A^2/(n-1)) = 4A^2 / (2A^2/(n-1)) = 2(n-1)$ which is approximately $n_t + n_c - 2$ — the same as Student's t for equal variance. When $A \gg B$ (treatment has much higher variance), the denominator is dominated by $A^2/(n_t-1)$, so $df \sim (n_t-1)$ — the effective sample size shrinks to the noisier group. This correctly makes the test more conservative when one group is much noisier: a small noisy group should not give you the same confidence as a large clean group.

Q: 7. What is minimum detectable effect and how does TrialCheck compute whether a test is underpowered?

Minimum Detectable Effect (MDE) is the smallest true effect size that a given experiment design (n , α , power) has 80% probability of detecting as statistically significant. TrialCheck's min sample size check inverts this: given the observed effect size and the configured alpha and power, it computes the required sample size. For proportions: $n_{\text{required}} = (z_{\alpha/2} + z_{\text{power}})^2 * 2 * p_{\text{bar}} * (1 - p_{\text{bar}}) / \delta^2$ where δ is the observed lift, $z_{\alpha/2}$ is the critical value at $\alpha/2$, and z_{power} is the value for the target power (0.84 for 80% power). If $n_{\text{actual}} < n_{\text{required}}$: WARN. The observed lift may be noise — the experiment had insufficient power to reliably detect an effect that small.

Q: 8. What is the novelty effect check — when would you trust it and when not?

The novelty effect check flags experiments where $\text{runtime} < \text{novelty_effect_days}$ (default 7). The concern: users who encounter a new feature may behave differently simply because it is new — clicking on new buttons out of curiosity, not because the feature is useful. This early-phase behaviour does not persist post-launch. Trust it when: the product is a high-engagement consumer app with rapidly changing user behaviour (social feeds, recommendation UI changes). Do not trust it when: (1) the product has infrequent user sessions (weekly-use SaaS) — 7 days may not be enough to even observe novelty effects; (2) the feature change is subtle enough that users are unlikely to notice it explicitly; (3) the experiment is for an operational metric (latency, error rate) rather than user behaviour.

Q: 9. Why no Bonferroni correction built in? When would that be wrong?

TrialCheck does not automatically apply Bonferroni because the right correction depends on the experiment design, not the number of metrics being checked. Bonferroni would be wrong when: (1) Metrics are correlated — Bonferroni assumes independence between tests. Revenue per user and conversion rate are correlated; Bonferroni overcorrects. (2) You have one pre-specified primary metric — Bonferroni is designed for exploratory analysis across many metrics, not for a single pre-specified hypothesis. (3) The guardrail metrics are directional — Bonferroni applies to two-tailed tests; guardrails are one-tailed (only the bad direction matters). Teams that want correction should reduce alpha before passing to TrialCheck, using the correction appropriate for their experiment design.

Q: 10. How does TrialCheck integrate with Statsig or Optimizely in practice?

TrialCheck accepts a plain ExperimentSummary dataclass. Integration is a thin wrapper layer that maps platform-specific API output to the ExperimentSummary schema. For Statsig: pull the experiment result via Statsig's REST API or SDK, extract `n_control`, `n_treatment`, `conversions`, `metric values`, `guardrail metrics`, map to ExperimentSummary, call `TrialCheck(experiment).run()`. The same pattern works for Optimizely, Split.io, or any internal platform. TrialCheck does not depend on any platform SDK — the integration lives entirely in the caller's wrapper, not in TrialCheck itself.

6.3 Testing, Limitations, and Positioning (Q11–Q15)

Q: 11. What is the difference between TrialCheck and just running a t-test in Python?

Running a t-test answers one question: is the mean difference significant? TrialCheck answers nine questions simultaneously: (1) Was randomisation correct (SRM)? (2) Were groups balanced before the test (pre-period balance)? (3) Is the primary binary metric significant (z-test)? (4) Is the continuous metric significant (Welch's t)? (5) Was the test called early (peeking)? (6) Is the lift large enough to matter (practical significance)? (7) Did any guardrail regress (guardrail movement)? (8) Was the experiment adequately powered (min sample size)? (9) Is early lift explained by novelty (novelty effect)? A t-test skipping SRM check is like calculating mean calories in a recipe that might have a mis-measured ingredient. TrialCheck validates the ingredient first.

Q: 12. What happens if $n_{\text{control}} = 0$ is passed in?

The SRM check would compute χ^2_{sq} using $n_{\text{exp_c}}$ based on total_n and planned ratio, producing a large χ^2_{sq} and definite FAIL. The primary metric check would compute $p_c = \text{conversions_c} / n_c$ — with $n_c = 0$ this is a division by zero. TrialCheck handles this with a guard: if $n_{\text{control}} \leq 0$ or $n_{\text{treatment}} \leq 0$, the primary and continuous metric checks return INSUFFICIENT_INPUT rather than raising ZeroDivisionError. The report notes 'variant has zero observations; cannot compute metric significance.' The SRM check still fires FAIL.

Q: 13. How would you extend TrialCheck to handle multi-armed experiments?

Multi-arm (3+ variants) requires: (1) SRM: extend χ^2_{sq} to sum over all k arms: $\sum_i (n_i - n_{\text{exp_i}})^2 / n_{\text{exp_i}}$, $df = k-1$. The erfc formula is for $df=1$ only; use the χ^2_{sq} CDF for $df>1$. (2) Primary metric: pairwise z -tests between each treatment arm and control. Apply multiple comparison correction (Bonferroni with $k-1$ comparisons, or Holm-Bonferroni). (3) Pre-period balance: compute SMD for each arm vs control. (4) Guardrails: check each arm independently. The ExperimentSummary dataclass would need to accept a List[VariantSummary] instead of a single treatment variant. This is a tracked roadmap item.

Q: 14. What is a guardrail metric and why does TrialCheck treat it differently?

A guardrail metric is a metric the experiment must not harm — latency, error rate, revenue per user in engagement experiments, or safety metrics. Unlike primary metrics, which you want to improve, guardrails have a direction constraint: only movement in the bad direction matters. TrialCheck treats guardrails differently: (1) No statistical test — guardrail movement is checked against a tolerance band, not tested for significance. A 3% latency increase is bad regardless of p -value. (2) Direction-aware — you specify 'increase' or 'decrease' as the bad direction. (3) WARN not FAIL — a guardrail regression is a tradeoff decision, not a statistical invalidity. The product team may decide to ship accepting the regression.

Q: 15. Walk me through an experiment that would pass all 7 checks — give me real numbers

Experiment: checkout redesign, planned 14-day run, 50/50 split, conversion metric. Assignment: $n_{\text{control}}=50,005$; $n_{\text{treatment}}=49,995$ (total=100,000). SRM: $\chi^2_{\text{sq}}=(5^2/50000)+(5^2/50000)=0.00001+0.00001=0.00002 \rightarrow p=0.996 \rightarrow \text{PASS}$. Pre-period balance: SMD on prior_revenue_per_user = 0.05 (<0.1) $\rightarrow \text{PASS}$. Primary metric: conversions_c=1,500(3.0%), conversions_t=1,615(3.23%). z-test gives $p=0.031$ (<0.05), lift=0.23pp=7.7% relative $\rightarrow \text{PASS}$ (significant). Peeking: actual_duration=14 days = planned_duration. interim_looks=0 $\rightarrow \text{PASS}$. Practical significance: 7.7% relative lift > 1% threshold $\rightarrow \text{PASS}$. Guardrail: p99_latency control=234ms, treatment=236ms (0.85% increase < 2% tolerance) $\rightarrow \text{PASS}$. Min sample size: observed 3% $\rightarrow$ 3.23%, $n_{\text{required}} \sim 4,800$ per arm, $n_{\text{actual}}=50,000 \rightarrow \text{PASS}$. Novelty: 14 days > 7-day novelty threshold $\rightarrow \text{PASS}$. Overall: PASS. Ready for ship decision.

6.4 Deep Dives — Edge Cases and Extensions (Q16–Q22)

Q: 16. How are check results aggregated into the overall status?

FAIL > WARN > PASS. If any single check returns FAIL: overall = FAIL, immediately. If no FAIL and any WARN: overall = WARN. If all checks return PASS: overall = PASS. INSUFFICIENT_INPUT is returned when required inputs are missing ($n=0$, no duration data for peeking, no guardrails for guardrail check). The report lists all checks individually so teams can see which specific checks failed, not just the aggregate status.

Q: 17. How does TrialCheck connect to MetaSignal in the portfolio?

MetaSignal is the full experimentation platform: it includes CUPED variance reduction, A/A calibration (1,000-run simulation to verify FPR), mSPRT sequential testing, and guardrail-first decisioning. TrialCheck is the post-hoc audit tool. Workflow: MetaSignal runs the experiment with proper statistical safeguards, generates the experiment summary, TrialCheck audits that summary before the team presents results. MetaSignal's CUPED reduces variance, potentially allowing TrialCheck to detect the lift with a smaller required sample size.

Q: 18. What is the erfc function and why use it instead of looking up a chi-sq table?

$\text{erfc}(x) = (2/\sqrt{\pi}) \cdot \int_x^\infty \exp(-t^2) dt = 1 - \text{erf}(x)$, the complementary error function. For a chi-squared distribution with $df=1$: $p = \text{erfc}(\sqrt{\text{chi_sq}/2})$. This is exact (not an approximation) and available as `math.erfc` in Python stdlib. Using `erfc` instead of a chi-sq table: (1) exact for all `chi_sq` values, not interpolated; (2) no external dependency (`scipy.chi2.sf` not required); (3) reproducible across all Python versions. The equivalence: $\text{chi2}(df=1).sf(x) = \text{erfc}(\sqrt{x/2})$ is a mathematical identity that TrialCheck's test suite verifies against `scipy`.

Q: 19. How do you set the practical significance threshold for a new product?

The threshold should be set before the experiment runs, based on business analysis: (1) What is the minimum lift that would justify the engineering and maintenance cost of the feature? For a feature that costs 2 engineer-weeks, what conversion improvement pays for that? (2) What is the noise floor — the smallest lift you can reliably detect with your typical experiment size? Setting the threshold below your noise floor creates ambiguity between 'practically significant but undetectable' and 'no effect'. (3) What are competitor benchmarks? A 0.1% conversion lift might be transformative at one business and noise at another. Default 1% relative lift is a conservative starting point; adjust per product context.

Q: 20. What happens when both peeking and practical insignificance fire?

Both produce WARN. The overall status is WARN (not worse — WARN is the max for these checks). The `TrialReport` includes both `CheckResults` in its checks list, each with its own detail, recommendation, and evidence. The interpretation string summarises all findings: 'Experiment was called at 50% of planned duration with 2 interim looks and the observed lift (0.3%) is below the practical significance threshold (1%). These findings together suggest the experiment should not ship.' TrialCheck never makes the ship decision — it documents the risk combination.

Q: 21. How would you test the SRM detection is correct?

The test suite validates against `scipy`: `from scipy.stats import chi2_contingency, p_scipy, _, _`
`= chi2_contingency([[n_c, n_t], [n_exp_c, n_exp_t]])` Verify `abs(p_trialcheck - p_scipy) < 1e-9` for multiple SRM scenarios. Test cases in the suite include: (1) Perfect 50/50 split -> PASS; (2) 56/44 split with total_n=100K -> FAIL; (3) Borderline 51/49 with small n -> PASS (within noise); (4) Non-50/50 planned ratio (e.g., 80/20 holdout) with correct observed split -> PASS.

Q: 22. What is the minimum experiment that TrialCheck can audit?

Minimum required fields: `n_control`, `n_treatment`, `expected_control_ratio`. With only these: SRM check runs. Everything else returns `INSUFFICIENT_INPUT`. Adding `conversions_c`, `conversions_t`: primary metric check runs. Adding `mean_c`, `mean_t`, `std_c`, `std_t`: continuous metric check runs. Adding `planned_duration_days`, `actual_duration_days`: peeking check runs. Adding guardrails list: guardrail check runs. TrialCheck is designed to degrade gracefully — each check runs if its required inputs are present and returns `INSUFFICIENT_INPUT` otherwise.

6.5 Advanced Topics — Scalability, CI Integration, Roadmap (Q23–Q27)

Q: 23. How does TrialCheck handle the case where treatment variance is much higher than control?

This is exactly the scenario Welch's t-test is designed for. High treatment variance (revenue experiment: high-spenders in treatment have higher std) means $A = s_t^2/n_t \gg B = s_c^2/n_c$. The Welch-Satterthwaite df formula then produces a smaller df, closer to $n_t - 1$ than $n_t + n_c - 2$. Smaller df $\rightarrow$ fatter t-distribution tails $\rightarrow$ higher p-value for the same t-statistic. This is the correct conservative behaviour: high treatment variance means more uncertainty about the true treatment mean, and the test should reflect that.

Q: 24. What is the guardrail tolerance parameter and how should it be set?

`guardrail_tolerance` is the maximum acceptable relative change in a guardrail metric before WARN fires. Default 2% (0.02). Both conditions must hold: movement in bad direction AND exceeds tolerance. Setting it: (1) Set based on SLO/SLA — if your p99 latency SLO is 250ms and baseline is 234ms, a 2% increase (4.7ms) takes you to 238.7ms, still within SLO. Adjust tolerance to reflect how much headroom you have. (2) Set based on measurement noise — if your metric has 1% measurement variance, a 0.5% tolerance would trigger on noise. (3) Set based on user impact — for safety metrics, tolerance may be 0 (any degradation is WARN).

Q: 25. How would you use TrialCheck in a CI/CD pipeline?

After each experiment concludes: (1) Export experiment summary from your platform (Statsig/Optimizely/internal) as JSON. (2) Load into ExperimentSummary via `load_experiment_json()`. (3) Run `TrialCheck(experiment).run()`. (4) Write report to JSON artifact. (5) Exit with non-zero code if `report.overall_status == CheckStatus.FAIL`. This creates an automated gate: SRM failures block the ship pipeline. WARN findings generate a Slack/email alert for human review. PASS findings auto-approve the experiment for the next review stage. The claim boundary in every report ensures no team misinterprets PASS as 'auto-ship approved'.

Q: 26. What would a false negative from TrialCheck look like?

The most dangerous false negative is a missed SRM. This would happen if: (1) The SRM is in a dimension not captured by the assignment counts — e.g., geographic SRM (50/50 overall but 80/20 in mobile users) when TrialCheck only receives aggregate counts. (2) A subtle assignment bug affects only high-value users, changing the assignment ratio by only 0.5% — not enough to trigger the chi-sq threshold. Other false negatives: peeking check misses peeking that was not documented (`interim_looks=0` even if peeking occurred), or guardrail regression below tolerance threshold that is still user-impactful.

Q: 27. How do the four demo scenarios cover the full decision space?

Scenario 1 (`clean_pass`): All 9 checks pass. 50/50 split, 14-day run, significant primary metric, lift > 1%, no guardrail movement. Shows what good looks like. Scenario 2 (`srm_fail`): 56/44 observed vs 50/50 planned. FAIL. Shows why every subsequent metric is suspect — the groups are not comparable. Scenario 3 (`peeking_warn`): 36% of planned duration, 2 interim looks. Primary metric significant ($p=0.018$) but peeking risk WARN. Shows that $p<0.05$ is not enough. Scenario 4 (`guardrail_harm`): Primary metric +4.1% lift. Revenue/user -4.4%, refund rate doubles. WARN. Shows the tradeoff conversation TrialCheck surfaces. Together: blocked (FAIL), hold-for-review (WARN), and clean-ship (PASS) cases.

6.6 Stress Tests — Multiple Arms, Statistics Literature (Q28–Q32)

Q: 28. Walk me through the two-proportion z-test — when does TrialCheck use it?

TrialCheck uses the two-proportion z-test when the primary metric is binary (e.g., conversion rate, click-through rate, activation rate). Formula: $z = (p_t - p_c) / SE_{\text{pooled}}$ where $SE_{\text{pooled}} = \sqrt{p_{\text{pooled}} * (1 - p_{\text{pooled}}) * (1/n_t + 1/n_c)}$ and $p_{\text{pooled}} = (\text{conversions}_t + \text{conversions}_c) / (n_t + n_c)$. Worked example: control $n=50,120$ conversions=1,500 $\rightarrow p_c=0.0299$. Treatment $n=50,480$ conversions=1,560 $\rightarrow p_t=0.0309$. $p_{\text{pooled}} = 3060/100600 = 0.0304$. $SE = \sqrt{0.0304 * 0.9696 * (1/50120 + 1/50480)} = 0.000766$. $z = (0.0309 - 0.0299) / 0.000766 = 1.306$. $p = 0.19$ (not significant). Use Welch's t-test for continuous metrics (revenue/user, session length, latency).

Q: 29. What is Standardised Mean Difference (SMD) and why does TrialCheck use it for pre-period balance?

$SMD = (\text{mean}_{\text{treatment}} - \text{mean}_{\text{control}}) / \text{pooled_std}$ where $\text{pooled_std} = \sqrt{(\text{var}_t + \text{var}_c) / 2}$. SMD measures pre-experiment baseline difference in the same units as Cohen's d. $SMD = 0$ means identical baselines. $SMD = 0.1$ means the groups differ by 10% of a pooled standard deviation. TrialCheck warns at $SMD > 0.1$ for any pre-period baseline metric. Why SMD instead of a t-test on the pre-period? A t-test depends on sample size: with $n=100,000$ per arm, even trivial $SMD=0.01$ would produce $p<0.001$. SMD is size-independent — it measures the magnitude of the imbalance, not whether it is statistically detectable. A well-randomised large experiment should have SMD near 0 regardless of sample size.

Q: 30. How does TrialCheck handle an experiment with multiple treatment arms?

ExperimentSummary supports multiple VariantSummary objects via the variants list. TrialCheck runs all checks for each (control, treatment_i) pair independently. SRM is checked across all arms simultaneously — the chi-sq test extends naturally to k arms with $df = k-1$. The practical implication: a 3-arm experiment (control, T1, T2) has two separate statistical tests (control vs T1, control vs T2) and one combined SRM check. Multiple comparison correction is explicitly NOT in scope for v0.3.0 — TrialCheck does not apply Bonferroni or Benjamini-Hochberg correction. This is documented in the claim boundary. For experiments with >3 metrics, multiple comparison correction is a critical missing piece.

Q: 31. What does TrialCheck output when there are no warnings or failures?

A PASS report includes: (1) `overall_status = CheckStatus.PASS`, (2) a finding for each of the 9 checks with status PASS, (3) the claim boundary text, (4) a plain-English interpretation: 'All 9 audit checks passed. No structural issues detected. This does not constitute a ship recommendation — consult domain experts for business context and practical significance.' The claim boundary is non-suppressible even on PASS. The reason: a PASS report with no claim boundary could be misread as 'TrialCheck approved this experiment for shipping.' TrialCheck cannot approve experiments — it can only flag problems. A PASS means no problems were found, not that none exist.

Q: 32. How does TrialCheck connect to the applied statistics literature?

Every implemented check maps to published methodology: (1) SRM: Fabijan et al. 2019 'Diagnosing Sample Ratio Mismatch in Online Controlled Experiments' (KDD). Used chi-sq goodness-of-fit, same as TrialCheck. (2) Peeking: Johari et al. 2015 'Peeking at A/B Tests' (SIGKDD). Quantifies FPR inflation with interim looks. (3) Welch's t-test: Welch 1947; Ruxton 2006 for defence of Welch over Student's. (4) SMD: Cohen 1988 for effect size; Austin 2011 for pre-experiment balance. (5) Minimum sample size: standard power analysis with $\alpha=0.05$, $\text{power}=0.8$. Connecting the implementation to published work demonstrates that the checks are grounded in experimental science, not ad hoc heuristics.

S7 Failure Modes

Failure Mode	Trigger	Impact	Mitigation
Missed SRM on subgroup	Aggregate n_c/n_t looks balanced but mobile users are 80/20	False PASS on SRM	Run SRM check on stratified subgroups; provide segment-level n
Peeking not documented	$interim_looks=0$ even if peeking occurred	False PASS on peeking check	Mandate $interim_looks$ logging at experiment creation
Welch df overflow for large n	Very large n with equal variances gives $df = n_t + n_c - 2 \rightarrow$ same as Student's	No impact on correctness; Welch is exact	Not a bug; Welch converges to Student's t when variances are equal
Guardrail below tolerance but user-impactful	3% latency increase but tolerance set to 5%	False PASS on guardrail check	Set tolerance based on SLO headroom, not arbitrary percentage
$n_{control} = n_{treatment} = 0$	Empty experiment passed in	Division by zero in metric checks	Guards return INSUFFICIENT_INPUT for $n \leq 0$; SRM still fires FAIL
Novelty threshold miscalibrated	Long-cycle SaaS: 7-day novelty window is too short	False PASS on novelty check	Adjust $novelty_effect_days$ per product's user session frequency

S8 Scalability

TrialCheck operates on ExperimentSummary objects — aggregated statistics, not raw user-level data. All checks are $O(1)$ with respect to experiment size: they operate on summary statistics (n , mean, std, conversions), not on individual observations. This means TrialCheck is inherently scalable for any experiment size.

Check	Input	Complexity	Note
SRM	n_c, n_t, ratio	$O(1)$	Two additions, one erfc call
Pre-period balance	k covariates (mean, std, n)	$O(k)$	k = number of covariates provided
Primary metric	$n_c, n_t, \text{conversions}_c, _t$	$O(1)$	One z-test computation
Continuous metric	n , mean, std per arm	$O(1)$	One Welch t-test
Peeking risk	duration, interim_looks	$O(1)$	Two comparisons
Practical sig	lift, threshold	$O(1)$	One comparison
Guardrail movement	m guardrail metrics	$O(m)$	m = number of guardrails
Min sample size	MDE, alpha, power	$O(1)$	One formula evaluation
Novelty effect	actual_duration_days	$O(1)$	One comparison

If you needed to audit 10,000 experiments per day: Each TrialCheck call runs in milliseconds (all $O(1)$ or $O(k)$ for small k). 10,000 audits per day = one per 8.6 seconds. Trivially parallelisable. The bottleneck would be fetching experiment summaries from the platform API, not TrialCheck computation.

8.2 Batch Audit Architecture — If Scale Were Required

TrialCheck's current design is single-experiment: one ExperimentSummary per call. If an organisation needed to audit hundreds of experiments per hour, the natural extension is a batch audit layer wrapping TrialCheck. Because all checks are $O(1)$, the concurrency model is embarrassingly parallel: each experiment is independent.

Proposed batch architecture:

```
class BatchAuditor: def __init__(self, experiments: List[ExperimentSummary],
max_workers: int = 8): self.experiments = experiments self.max_workers = max_workers
def run_all(self) -> List[TrialReport]: from concurrent.futures import
ThreadPoolExecutor with ThreadPoolExecutor(max_workers=self.max_workers) as pool:
futures = [pool.submit(TrialCheck(e).run) for e in self.experiments] return
[f.result() for f in futures] def summary_table(self, reports: List[TrialReport]) ->
```

```
str: rows = [(r.experiment_id, r.overall_status.value, len([c for c in
r.check_results if c.status == CheckStatus.WARN])) for r in reports] return
tabulate(rows, headers=['experiment', 'status', 'warns'])
```

This design is $O(n)$ with a constant-factor improvement from `ThreadPoolExecutor`. The GIL is not a bottleneck here because `TrialCheck` computations are CPU-light (arithmetic on small numbers). At 1,000 experiments per batch with `max_workers=8`, wall time is approximately $\text{total_experiments} / \text{workers} * \text{per_experiment_ms}$. At 5ms per experiment: $1,000 \text{ experiments} / 8 \text{ workers} = 125 \text{ per worker} * 5\text{ms} = 625\text{ms}$.

Why batch mode is not built in v0.3.0: The primary use case is a DS auditing one experiment before presenting results, not a pipeline processing thousands. Batch mode would require experiment ID tracking, partial failure handling, and output aggregation — complexity not justified by the current use case. Adding batch mode is a tracked v1.0 roadmap item.

8.3 System-Level Thinking for Interview

A FAANG interviewer asking about scalability wants to know if you can reason about the system's bottlenecks. For `TrialCheck`, the relevant bottlenecks are:

Data fetch, not computation: Fetching experiment summaries from Statsig or Optimizely involves network calls to their APIs. The actual `TrialCheck` computation is microseconds. If you are auditing 100 experiments, 99% of wall time is API calls.

Parallelism model: `ThreadPoolExecutor` is appropriate because the GIL does not matter for I/O-bound work (API calls). If computation were the bottleneck, `ProcessPoolExecutor` would be needed. For `TrialCheck`'s use case, threads are correct.

Output storage: JSON reports are small (~2KB each). $10,000 \text{ reports/day} = 20\text{MB/day}$. Trivially storable in S3 or any blob storage. Indexing by experiment ID and timestamp enables audit trail queries.

Fail-fast on SRM: For a pipeline processing hundreds of experiments, an early-exit option on FAIL would be useful: skip the remaining checks if SRM fires. Current design runs all checks regardless. Both designs are defensible — the current one gives full visibility even on FAIL, which is more useful for diagnosis.

S9 Cross-Project Connections

TrialCheck is the experiment validity gate in the Applied LLM Systems portfolio. Understanding where it fits demonstrates system-level thinking about decision infrastructure.

Project	Relationship to TrialCheck
MetaSignal	MetaSignal is the full experimentation platform (CUPED, A/A calibration, mSPRT sequential testing). TrialCheck is the post-hoc audit tool. MetaSignal generates the experiment outputs; TrialCheck audits them.
PulseRank	PulseRank uses A/B simulation to validate NDCG improvements. Before accepting a HOLD_SIMULATED decision, TrialCheck validates the simulation has sufficient power, no SRM, and no multiple comparison inflation.
RiskFrame	A/B tests on ML threshold policy changes (e.g., approval rate v1.0 vs v1.1) pass through TrialCheck to confirm statistical significance before the policy is locked.
GoldenSetAuditor	TrialCheck validates experiment design; GoldenSetAuditor validates the evaluation dataset used in the experiment. Both gate the same model improvement decision from different angles.
DevPulse	DevPulse RAG versioning experiments can use TrialCheck to validate that the A/B comparison between RAG versions is statistically sound before the DevPulse conflict detection is trusted.

9.2 TrialCheck in the Full Experiment-to-Decision Pipeline

At FAANG, experiment-to-decision involves multiple stages. TrialCheck sits at the readout gate:

- Stage 1 — Experiment design:** Define hypothesis, primary metric, MDE, planned duration, alpha, power. Tools: power calculators, experiment planning docs. TrialCheck validates the result of this stage in check 8 (min sample size).
- Stage 2 — Experiment execution:** Assignment, instrumentation, data collection. Tools: Statsig, LaunchDarkly, in-house randomisation. TrialCheck validates this stage in check 1 (SRM) and check 2 (pre-period balance).
- Stage 3 — Readout gate:** TrialCheck. Run before anyone reads the results. Checks 1-9 catch the common failure modes. FAIL blocks readout; WARN requires documented human review before proceeding.
- Stage 4 — Statistical analysis:** Compute p-values, confidence intervals, effect sizes. TrialCheck runs the primary statistical tests (checks 3-4) but teams may do deeper analysis (stratification, CUPED, sequential testing) separately.

Stage 5 — Ship decision: Human decision with business context. TrialCheck's claim boundary explicitly states it does not make this decision.

Placing TrialCheck at Stage 3 catches methodology failures before they infect the statistical analysis and the ship decision. A FAIL at Stage 3 sends the experiment back to Stage 2 for investigation.

S10 Resume Claim Audit

Resume-safe claim (from README):

Built TrialCheck, a platform-agnostic A/B experiment readout auditor that checks completed experiment summaries for sample-ratio mismatch (chi-square), peeking risk, MDE context, practical and statistical significance (two-proportion z-test and Welch's t-test), guardrail movement, and pre-period covariate imbalance (SMD), producing JSON/Markdown/HTML audit reports with explicit decision caveats. Zero dependencies. 27 tests. 4 canonical demo scenarios.

Claim	Defensible?	Evidence
sample-ratio mismatch (chi-square)	YES	stats.py chi_square_df1_survival; validated vs scipy
peeking risk	YES	checks.py duration_ratio + interim_looks check
MDE context	YES	minimum_sample_size_check in checks.py
two-proportion z-test	YES	stats.py two_proportion_z_test
Welch's t-test	YES	stats.py welch_t_test; validated vs scipy
guardrail movement	YES	direction check + tolerance band in checks.py
pre-period covariate imbalance (SMD)	YES	SMD computation in checks.py
JSON/Markdown/HTML reports	YES	write_report() in io.py
explicit decision caveats	YES	claim boundary in every report, non-suppressible
Zero dependencies	YES	no runtime deps; scipy only in tests
27 tests	YES	tests/ directory in repo
4 demo scenarios	YES	scripts/generate_demo_reports.py

10.2 What You Cannot Claim

Do NOT claim: 'I built an experiment analysis platform' or 'I built a statistical testing framework.' TrialCheck is an audit tool, not a platform. It does not run experiments, it audits completed readouts. If asked 'how do you run the experiment?', the correct answer is: 'TrialCheck does not run experiments. It audits completed experiment summaries that your experimentation platform (Statsig, Optimizely, in-house) produces.'

10.3 What the Interviewer Is Testing

Statistical rigour: Do you know the difference between Welch's and Student's t-test and why Welch's is correct for A/B tests? Can you derive the chi-squared SRM statistic without looking it up?

Intellectual honesty: Do you proactively admit what TrialCheck does not do? CUPED, sequential testing, and multiple comparison correction are the three most obvious gaps — name them before being asked.

Systems thinking: Can you place TrialCheck in the context of a full experimentation pipeline? The 5-stage pipeline (design, execution, readout gate, analysis, decision) is the answer to 'where does this fit?'

Practical impact thinking: Can you quantify the cost of not running TrialCheck? The SRM example from S1.5 (3 months of downstream development on a false positive) is the go-to answer for 'why does this matter?'

10.4 Resume Bullet Options

Senior Engineer / DS: Built TrialCheck, a platform-agnostic A/B experiment readout auditor implementing 9 checks (SRM via chi-squared + erfc, Welch's t-test for continuous metrics, two-proportion z-test for binary metrics, peeking risk, pre-period SMD balance, guardrail movement, practical significance, MDE power analysis, novelty effect), with zero runtime dependencies and validated statistical implementations. 32 Q&A; pairs.

Mid-level: Built TrialCheck to package senior DS experiment review checklists into a callable library. Derived and implemented Welch-Satterthwaite df formula from first principles; validated against scipy as test ground truth. Platform-agnostic via ExperimentSummary dataclass. Connects to MetaSignal and GoldenSetAuditor in a 13-project portfolio.

S11 Study Plan + Do Not Say This

11.1 One-Week Study Plan

Day 1: Read `models.py` and `checks.py` start to finish. Understand the `ExperimentSummary` dataclass structure. Trace `TrialCheck(experiment).run()` through every check.

Day 2: Work through S3 (Algorithm Explanations). Derive the chi-sq SRM formula by hand. Compute the Welch t-statistic and Welch-Satterthwaite df for the revenue example in S3.2.

Day 3: Run the demo: `pip install trialcheck`; run `generate_demo_reports.py`; read all 4 scenario reports. Modify the `srm_fail` scenario to a borderline SRM (51/49).

Day 4: Practice Q&A; pairs from S6. Focus on the required 15. For each statistical question, derive the formula from memory before checking.

Day 5: Study design decisions in S4. Practice defending zero-dependency design and FAIL-reserved-for-SRM choices. Know why Welch not Student's.

Day 6: Read S7 (failure modes). Practice explaining what a false negative from `TrialCheck` looks like and why it matters.

Day 7: Mock interview. Derive SRM chi-sq from scratch. Walk through the full pass scenario from Q&A; #15 with real numbers. No notes.

11.1a What an E5 Interviewer Is Actually Testing

For a senior data scientist or ML engineer role at FAANG, a project discussion on `TrialCheck` tests five dimensions:

Statistical depth: Can you derive SRM chi-squared, Welch's t, and Welch-Satterthwaite df from first principles? Can you explain WHY Welch not Student's? If you cannot derive these, practice S3 until the formulas are mechanical.

System design thinking: Can you explain why `ExperimentSummary` is a platform-agnostic dataclass and not a database query? Can you describe how `TrialCheck` fits into a CI/CD pipeline alongside an experimentation platform?

Honest limitation awareness: Can you proactively state what `TrialCheck` does NOT do (CUPED, sequential testing, multiple comparison correction) and WHY those are excluded from v0.3.0?

Design trade-off reasoning: For every design decision in S4, can you state the alternative and explain the rejection reason? 'Welch not Student's because homoscedasticity is violated in A/B tests' is the format.

Practical impact: Can you give a concrete example of what would have gone wrong if TrialCheck had not been run? The SRM example in S1.5 is the go-to.

Drill tip: For Q3 (derive SRM from first principles), practice until you can write $\chi^2_{sq} = (n_c - n_{exp_c})^2 / n_{exp_c} + (n_t - n_{exp_t})^2 / n_{exp_t}$ on a whiteboard in under 30 seconds, then compute the concrete example (56/44 $\rightarrow \chi^2_{sq}=1440$, $p \approx 0$). This question is almost certain in any statistics-heavy interview.

11.1b Common Ambush Questions for TrialCheck

'Isn't this just experiment quality assurance that every platform already provides?' —

No. Platforms like Statsig and Optimizely show you experiment results; they do not audit methodology quality. They will display a p-value even if the experiment had SRM. They will not warn about peeking unless you use their sequential testing features. TrialCheck is a platform-independent methodological audit.

'Your peeking check is just two heuristics — that's not rigorous.' — Correct, and documented. The heuristics (duration ratio < 0.8, interim_looks >= 3) are conservative proxies. Rigorous peeking control requires always-valid sequential tests (mSPRT) which is on the v1.0 roadmap. The current check catches the most common and egregious cases. A WARN does not block the experiment — it prompts human review.

'How do you know your math is right?' — Every statistical implementation is validated against scipy reference values in the test suite. For SRM: against `scipy.stats.chi2_contingency`. For Welch t: against `scipy.stats.ttest_ind(equal_var=False)`. For z-test: against `scipy.stats.proportions_ztest`. The tests fail if TrialCheck's values diverge from scipy by more than $1e-6$.

11.2 Do Not Say This

NEVER SAY THIS	SAY THIS INSTEAD
"It runs the A/B experiment"	"It audits completed experiment readouts. It does not run experiments, assign users, or replace an experimentation platform."

NEVER SAY THIS	SAY THIS INSTEAD
"It decides whether to ship"	"It surfaces risks and documents them. The ship/no-ship decision is always human. TrialCheck is decision support, not a decision-maker."
"The t-test I used"	"Welch t-test specifically — not Student's t — because A/B test groups almost always have unequal variances due to the treatment effect itself."
"p < 0.05 means it worked"	"Statistical significance is necessary but not sufficient. SRM, peeking, practical significance, guardrail movement, and pre-period balance all matter independently."
"It uses scipy for the statistics"	"Zero runtime dependencies. All statistical tests are implemented from scratch using Python stdlib math.erfc, sqrt, etc. Scipy is used only in tests as ground truth."
"SRM means the split was wrong"	"SRM means the observed assignment ratio differs significantly from the planned ratio in a way that suggests non-random assignment. It invalidates all downstream tests because the groups are no longer comparable."

11.3 The 30-Second Elevator Pitch

TrialCheck is a platform-agnostic A/B experiment readout auditor — a Python library that checks completed experiment results for 9 common methodology failures before anyone reads out the results. It implements SRM detection via chi-squared goodness-of-fit, Welch's t-test for continuous metrics, two-proportion z-test for binary metrics, peeking risk, pre-period balance via SMD, guardrail movement, practical significance, minimum sample size, and novelty effect. Zero runtime dependencies — all statistics implemented from stdlib. Validated against scipy. 32 Q&A; pairs. `pip install trialcheck`.

11.4 Quick-Reference Formula Card

Formula	What It Computes	Used In
$\chi^2_{sq} = \sum (\text{obs}_i - \text{exp}_i)^2 / \text{exp}_i$	Chi-squared statistic for observed vs expected split	SRM detection
$p = \text{erfc}(\sqrt{\chi^2_{sq} / 2})$	Exact p-value for chi-sq with df=1 via complementary error function	SRM p-value
$t = (\text{mean}_t - \text{mean}_c) / \sqrt{s_t^2/n_t + s_c^2/n_c}$	Welch t-statistic for two-sample mean comparison	Continuous metric significance
$df = (A+B)^2 / (A^2/(n_t-1) + B^2/(n_c-1))$ where $A=s_t^2/n_t$, $B=s_c^2/n_c$	Welch-Satterthwaite degrees of freedom	Welch t-test df

Formula	What It Computes	Used In
$z = (p_t - p_c) / \sqrt{p_{\text{bar}}(1-p_{\text{bar}})(1/n_t + 1/n_c)}$	Two-proportion z-test statistic	Binary primary metric significance
$\text{SMD} = (\text{mean}_t - \text{mean}_c) / \sqrt{(\text{var}_t + \text{var}_c) / 2}$	Standardised mean difference for pre-period balance	Pre-period balance check
$\text{duration_ratio} = \text{actual_duration} / \text{planned_duration}$	Fraction of planned experiment duration completed	Peeking check (warn if < 0.8)

TrialCheck Interview Defense v3.0 | Sidharth Kriplani | 32 Q&A; Pairs | FAANG Standard
github.com/SidharthKriplani/trialcheck | `pip install trialcheck`

SA Glossary of Terms Used in This Document

Term	Definition as Used in TrialCheck Context
A/B experiment	A controlled experiment that randomly assigns users to a control or treatment group and measures the effect of a change on a metric.
SRM (Sample Ratio Mismatch)	Observed assignment ratio differs from planned ratio in a way unlikely under correct randomisation. Detected via chi-sq goodness-of-fit test.
Peeking	Calling an experiment significant after an interim look before the planned duration ends. Inflates effective false positive rate beyond alpha.
Welch's t-test	Two-sample t-test that does not assume equal group variances. Uses Welch-Satterthwaite df formula. Preferred over Student's t for A/B tests.
Two-proportion z-test	Test for equality of two proportions (conversion rates). Uses pooled standard error under the null hypothesis.
SMD (Standardised Mean Difference)	Effect size measure for pre-period baseline difference: $(\text{mean}_t - \text{mean}_c) / \text{pooled_std}$. Size-independent measure of group imbalance.
ExperimentSummary	Platform-agnostic Python dataclass containing all fields needed to run all 9 TrialCheck checks: assignment counts, metrics, guardrails, duration, config.
VariantSummary	Dataclass representing one arm (control or treatment): name, n, conversions, mean, std, and optional pre-period covariates.
GuardrailMetric	Secondary metric with a declared bad direction and tolerance band. Fails gracefully if movement exceeds tolerance.
Claim boundary	Non-suppressible text in every report stating that TrialCheck does not make ship/no-ship decisions. Analogous to GoldenSetAuditor's truth boundary.
Practical significance	Whether the observed lift meets the business's minimum meaningful change threshold, independent of statistical significance.
Novelty effect	Initial engagement spike when users first encounter a new feature, which inflates early metrics. Warn if experiment runtime < novelty washout window.
erfc	Complementary error function from Python math stdlib. Used for exact p-value computation: $p = \text{erfc}(\sqrt{\text{chi_sq}/2})$ for SRM.